Document number:	P3582R0
Date:	2025-01-13
Audience:	SG21, EWG
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Observed a contract violation? Skip subsequent assertions!

In this paper we propose a modification (on top of [P2900R12]) to contract assertion evaluation semantic *observe* in order to avoid undefined behavior that could otherwise be introduced in certain scenarios.

1. Motivation

Consider the following function definition.

```
Tool* selectTool(Tool* ta, Tool* tb)
  pre (ta != nullptr)
  pre (ta->is_configured())
  pre (tb != nullptr)
  pre (tb->is_configured())
  post (r: r != nullptr)
  post (r: r->is_configured())
{
  if (preferFirst) return ta;
  else             return tb;
}
```

This function has a set of preconditions. The function can execute without undefined behavior (UB) even when the pointers passed are null. The only reason we express strong preconditions here is that we want to guarantee a strong postcondition.

Now, consider the following scenario. A program that has been using this function in production for years and performs satisfactorily may be calling it with null pointers. This has never surfaced because other parts of the program do not take advantage of the postcondition. Next, a library author decides to introduce contract annotaitons at some point, and recommends to the users that they first build the program with the new (that is, all) assertions observed rather than enforced. If we hit a situation where `ta` is null, we end up with a newly introduced UB (when evaluating predicate `ta->is_configured()`) that was not present in the original program.

Note that this UB does not happen when contract assertions are evaluated in *ignore* semantic (because the offending expression is not evaluated) or in the *enforce* semantic (because we terminate before we reach the offending expression). This is a problem exlusive to the *observe* semantic.

2. The proposal

We prpose that in any of the following sequences:

- caller-facing preconditions,
- callee-facing preconditions,

- caller-facing postconditions,
- callee-facing postconditions,
- assertion statements,

when a given assertion is evaluated with *observe* semantics, and the contract violation is observed, the subsequent assertions in the sequence are evaluated with *ignore* semantics (or simply not evaluated).

There is one negative consequence of this proposal. The goal of obtaining unintrusive feedback about contract violations in a stable program is impeded. Consider a different function:

```
void fun (int x, int y)
  pre (x != 0)
  pre (y != 0);
```

With *observe* semantics in place, the program execution could report all violations of both predicates. But with this proposal, the program will never report that `y` is zero when `x` is zero.

The difference between the two examples is that in the first case the two predicates are *related* while in the latter they are not. We know of no way of discriminating the two cases, so this proposal favours avoiding UB in the former example, while compromising the use case using the latter example.

3. Acknowledgements

Tomasz Kamiński has suggested this solution to the introduced UB problem.

5. References

- [P2900R12] — Joshua Berne, Timur Doumler, Andrzej Krzemieński et al., "Contracts for C++", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2900r12.pdf").